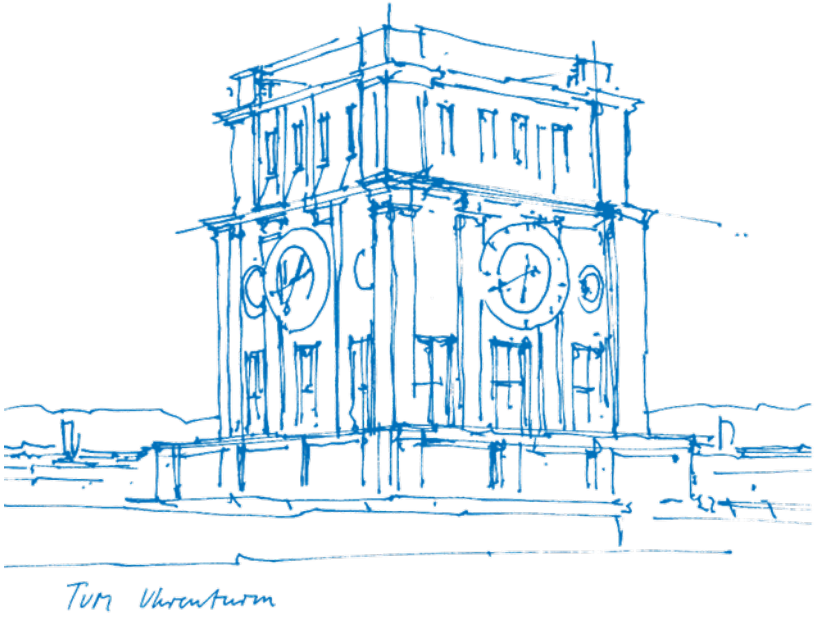
Repetitorium - WS25/26

Patrick Bokelmann, Maxi Heller, Justin Lanfermann, Alisa Parashchenko, Jaden Rotter, Nathan Schmidt, Leon Stark, Isabella Tefera

Grundlagen: Betriebssysteme und Systemsoftware (IN0009/IN0034)

Lehrstuhl für Connected Mobility
School of Computation, Information and Technology
Technische Universität München

Garching, 31.01.2026



Disclaimer:

Bei allen Folien handelt es sich um privat erstellte und verwaltete Folien für GBS im WS25/26.

Sie erheben keinerlei Anspruch auf Vollständigkeit und Korrektheit.

Im Zweifel haben IMMER die offiziellen Vorlesungsmaterialien recht!

© F. Sauter & C. Menges & S. Kappes & J. Schönberger & D. Rentz & A. Stephan & P. Bokelmann & T. Jungmann & N. Schmidt & J. Rotter & A. Scheitler & N. Haunschild & M. Heller & J. Lanfermann & A. Parashchenko & L. Stark & I. Tefera

Synchronisation

Parallele Systeme: Synchronisation

- Abstimmung des zeitlichen Verhaltens nebenläufiger Aktivitäten
- Koordinierung von Zugriffen auf gemeinsame, exklusiv nutzbare Ressourcen

- Race Condition
  - Mindestens zwei Prozesse greifen lesend oder schreibend auf gemeinsame Ressourcen zu
  - Das Ergebnis hängt von der Reihenfolge der Prozess-Ausführung ab

- Wechselseitiger Ausschluss (engl. mutual exclusion) von konkurrierenden Aktionen zur Vermeidung von Race Conditions

Kritischer Abschnitt

- (engl. critical section, critical region)
  - In Kritischen Abschnitten greifen Prozesse auf gemeinsame Ressourcen zu
  - Der Zugriff muss koordiniert werden, um Race Conditions zu vermeiden
  - Bei exklusiv nutzbaren Ressourcen wird der Zugriff sequentialisiert
- Beispiel: Prozesse mit kritischen Abschnitten

```
/* P0 */
int main(void) {
    ...
    enter_crit_region();
    x = x + 1;
    leave_crit_region();
    ...
}
```

```
/* P1 */
int main(void) {
    ...
    enter_crit_region();
    x = x + 2;
    leave_crit_region();
    ...
}
```

Beispiel: Erzeuger-Verbraucher

```
void producer (void) {
    int item = 0;

    while (true) {
        item = produce_item ();
        if (count == N)
            sleep ();
        insert_item (item);
        count = count + 1;
        if (count == 1)
            wakeup (consumer);
    }
}
```

```
void consumer (void) {
    int item = 0;

    while (true) {
        if (count == 0)
            sleep ();
        item = remove_item ();
        count = count - 1;
        if (count == N-1)
            wakeup (producer);
        consume_item (item);
    }
}
```

Pseudocode oben ist fehlerhaft: Was ist das Problem?

Wechselseitiger Ausschluss

- Anforderungen an die Realisierung des wechselseitigen Ausschlusses:
  1. Die kritischen Abschnitte sind wechselseitig ausgeschlossen
  2. Eine Realisierung des w. A. darf nicht von einer Annahme über die Reihenfolge, in der die Prozesse ihre kritischen Abschnitte ausführen, abhängen
  3. Eine Realisierung des w. A. darf nicht von Annahmen über die Ausführungszeit der Prozesse abhängen
  4. Kein Prozess darf einen anderen Prozess unendlich lange daran hindern, seinen kritischen Abschnitt auszuführen

- Grundlage für Lösungen
  - Atomare, d.h. nicht teilbare Operationen
  - Atomare Operationen können von Hard- und/oder Software zur Verfügung gestellt werden

Semaphore

- Problem: fehleranfällige Nutzung von low-level Diensten
- Lösung: BS-Konzept auf höherem Abstraktionsniveau
  - Realisierung von Semaphoren mit aktivem und passivem Warten möglich
- Definition Semaphore (E.W. Dijkstra, 1965)
  - Eine Semaphore (Signalmast) ist eine ganzzahlige Kontrollvariable s.
  - Auf der Kontrollvariable s sind 3 Operationen definiert: **Initialisierung**, **P** (down, wait) und **V** (up, signal).
  - Der Wert der Kontrollvariable s definiert die Anzahl der Prozesse, die gleichzeitig den kritischen Abschnitt betreten dürfen
- Operationen P (Proberen, *passering*) = down und V (Verhogen, *vrijgave*) = up sind atomar
  - P und V werden wechselseitig ausgeschlossen ausgeführt
  - Die Atomarität von P und V wird unter Nutzung von Hardware-Befehlen umgesetzt
  - Anmerkung: Die Kontrollvariable s ist ein zählendes Semaphor. Damit lassen sich allgemeine Synchronisationsprobleme lösen

Semantik der Semaphor-Operationen

- Operation down (bzw. P)
  - Kontrollvariable s wird dekrementiert
  - Falls s < 0, muss der Prozess warten
  - Bei einer Lösung mit passivem Warten
    - Prozess wird in Zustand wartend überführt (verwaltet in einer Wait-Queue)
- Operation up (bzw. V)
  - Die Kontrollvariable s wird inkrementiert
  - Bei passivem Warten: Falls die Wait-Queue nicht leer ist, wird ein Prozess aus der Queue in den Zustand rechenbereit überführt

```
void down (semaphore *wa) {
  int* s = &wa->s;
  *s -= 1;
  if (*s < 0) {
    thread_yield(wa->wait_queue);
  }
}
```

```
void up (semaphore *wa) {
  int* s = &wa->s;
  *s += 1;
  if (*s <= 0) {
    thread_wakeup(wa->wait_queue);
  }
}
```

Übungsaufgaben

Aufgabe 8 Semaphoren und Mutexe (9 Punkte)

Im Folgenden wird die parallele Ausführung der beiden nachfolgenden Threads untersucht. Nehmen Sie an, die Variablen `x` und `y` sind im Shared Memory beider Threads **initialisiert und auf 0 gesetzt worden**. Nehmen Sie weiterhin an, die Addition und Subtraktion in Zeile 3 seien je atomar.

```
1 // Thread 1
2 if (y >= 0) {
3     x = x + 1; // atomic
4 }
```

```
1 // Thread 2
2 while (x <= 0 && y > -3) {
3     y = y - 1; // atomic
4 }
```

0 1 2 3 4

a)* Was sind **alle** möglichen Kombinationen der Werte für `x` und `y` nach der nicht-deterministischen Ausführung der obenstehenden Threads? **Hinweis**: Die Länge der Tabelle muss nicht der Anzahl an Antworten entsprechen.

x									
y									

0 1 2 3 4 5

b)* Ergänzen Sie im Kasten unten im Initialisierungsblock die Deklaration und Initialisierung von Semaphoren und Mutexen. Ergänzen Sie anschließend unten im Codeblock dazugehörige `up()`- und `down()`-Aufrufe, sodass der oben gegebene nicht-deterministische Code immer in einem deterministischen Zustand terminiert, in dem **`y = –1` und `x = 1`**. Benutzten Sie **nicht mehr als 3** Semaphoren und Mutexe.

Initialisierungsblock

Codeblock

```
// Thread 1

if (y >= 0) {

    x = x + 1;

}
```

```
// Thread 2

while (x <= 0 && y > -3) {

    y = y - 1;

}
```

Aufgabe 5 Blätterfressen-Europameisterschaft (11 Punkte)

0 1 2 3 4 5 6 7

a)* Alle vier Jahre veranstalten die Faultiere die Blätterfressen-Europameisterschaft. Dieses Jahr hat die Stadt München den Zuschlag für das Eröffnungsspiel erhalten und beauftragt Sie mit der Leitung eines Ticket-Kiosks. Der Verkauf wird durch den unten abgebildeten Pseudocode beschrieben.
Durch die Funktion `TicketKaufen()` kann ein Faultier nach Betreten des Ticket-Kiosks aus allen gerade verfügbaren Tickets genau eines erwerben. Um ein neues Tickets bereitzustellen, kann der Ticket-Lieferant `TicketInVerkaufStellen()` ausführen. Nehmen Sie vereinfachend an, dass der Verkauf nie endet, die Funktionen unendlich oft aufgerufen werden können und das Stadion unendlich viele Sitzplätze besitzt.

Treffen Sie geeignete Synchronisationsmaßnahmen, indem Sie wo nötig **Semaphore** bzw. **Mutexe** sowie Aufrufe von `up()` und `down()` einführen. Unterbrechen Sie den Programmfluss nur so kurz wie möglich und verursachen Sie keine Deadlocks. Beachten Sie folgende Bedingungen (vgl. Abbildungen 5.1 und 5.2):

- Da es nur einen Verkäufer gibt, soll sich immer nur **höchstens ein Käufer-Faultier** am (anfangs freien) Verkaufskiosk aufhalten.
- Um zu vermeiden, dass sich Verfügbarkeiten ändern, während ein Faultier gerade seine Auswahl trifft, sollen neue Tickets nur zum Ticketstapel hinzugefügt werden, wenn sich **kein Käufer-Faultier** am Kiosk befindet.
- Die Stadt möchte, dass jederzeit **maximal 1000 Tickets** erwerblich sind. Anfangs ist keines verfügbar. Ist der Ticketstapel leer, so ist der Kiosk bis zur nächsten Auffüllung geschlossen.
- Aus Sicherheitsgründen sollen stets **nie mehr als 500 Faultiere** in einer (anfangs leeren) Schlange vor dem Kiosk warten.

```
// Globale Deklarationen
```

```
TicketKaufen() {

<Stelle in der Warteschlange an>

<Betrete den Ticket-Kiosk>

<Suche Stadion-Sitzplatz aus>

<Kaufe Ticket und verlasse Kiosk>

}
```

```
TicketInVerkaufStellen() {

<Generiere neues Ticket T>

<Drucke T aus>

<Füge T zu Ticketstapel hinzu>

}
```

b) Geben Sie für jede der drei noch nicht behandelten Deadlock-Bedingungen nach Coffman et al. (1971) an, ob sie in Ihrem zuvor **ergänzten** Pseudocode erfüllt ist oder nicht. Begründen Sie jeweils knapp anhand des Codes.

0 1 2 3

	Ja/Nein	**Begründung**
No-Preemption	Ja	
Mutual Exclusion		
Hold-and-wait		
Circular wait		

c)* Wie unterscheiden sich Mutexe von binären Semaphoren? Begründen Sie Ihre Antwort.

0 1

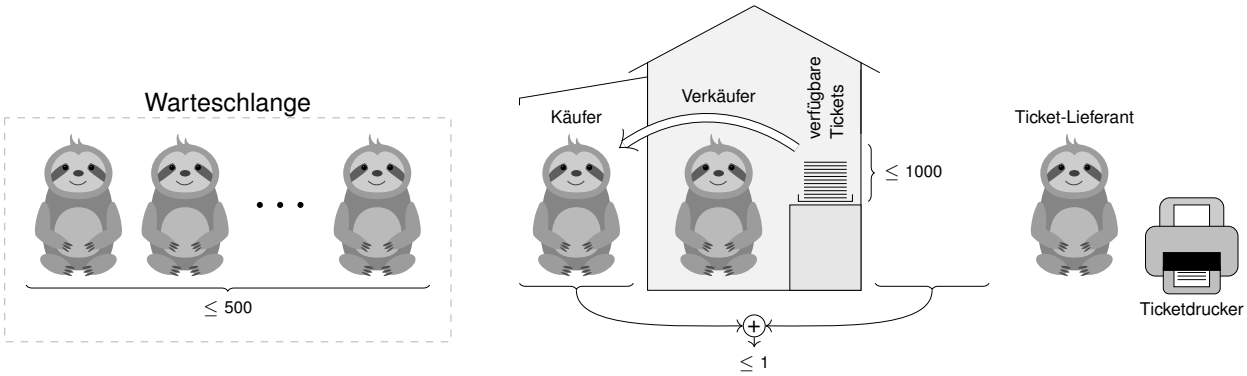


Abbildung 5.1: Grafische Veranschaulichung des Ticket-Verkaufs

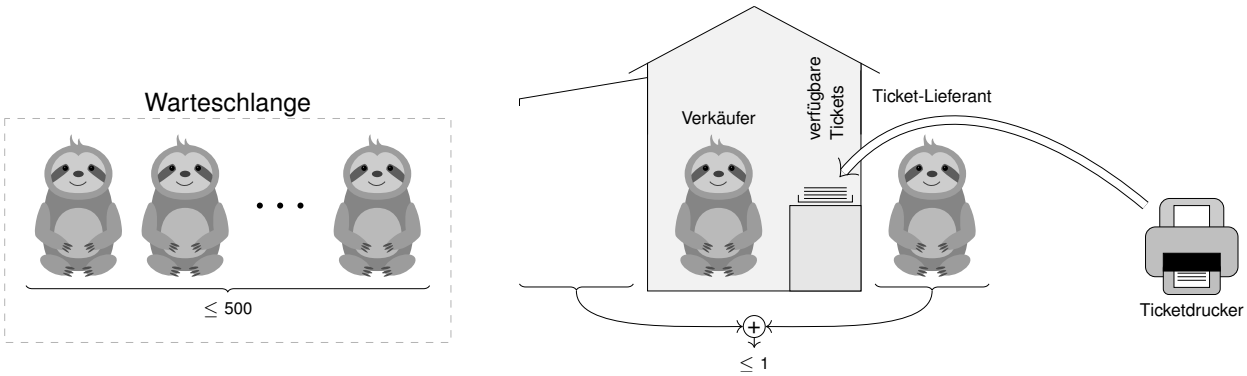


Abbildung 5.2: Grafische Veranschaulichung des Ticket-Auffüllens